Software Proposal: Spatiotemporal Graph Koopman Dynamics (Target: JOSS)

1. Executive Summary

The Koopman operator has revolutionized data-driven modeling of non-linear dynamical systems by providing a global linear representation in a high-dimensional functional space. However, existing open-source libraries predominantly treat state variables as flat vectors, completely ignoring the underlying spatial topology of the data. This proposal outlines the development of a novel open-source Python package module designed to seamlessly integrate Graph Neural Networks (GNNs) with Koopman operator theory. By doing so, we will enable the discovery of linear dynamics over spatiotemporal graph structures. This document outlines the software development plan with the explicit goal of publication in the *Journal of Open Source Software (JOSS)*.

2. Statement of Need (JOSS Criterion)

JOSS reviewers evaluate the software's academic utility, target audience, and the specific gap it fills in the current research ecosystem. While the theoretical concept of Graph Koopman Autoencoders (GKAE) has been recently explored in domain-specific literature, there is currently no standardized, general-purpose software package to implement it.

- **The Ecosystem Gap:** Current state-of-the-art packages like PyKoopman and DLKoopman require spatial data to be vectorized. Flattening a graph into a vector destroys topological inductive bias (e.g., permutation equivariance) and introduces an intractable *O(N^2)* scalability bottleneck for large networks.
- **Target Audience:** Researchers and data scientists analyzing networked dynamical systems (e.g., smart power grids, traffic networks, epidemiology) who currently must build PyTorch Geometric Koopman implementations entirely from scratch.

3. Proposed Software Architecture

We propose building a dedicated library that treats the observable lifting function as a localized message-passing operation.

3.1. Algorithmic Workflow

1. **Graph Encoding (The Lifting Function):** A Graph Convolutional Network (GCN) or Graph Attention Network (GAT) acts as the encoder, lifting physical node features into a high-dimensional latent space while respecting network topology.
2. **Linear Evolution (The Koopman Operator):** The latent state of the entire graph is advanced in time via a learned, finite-dimensional approximation of the Koopman operator matrix (K).
3. **Graph Decoding:** A structurally symmetrical GNN decodes the advanced latent state back into physical node predictions for the next timestep.

3.2. Proposed API Syntax

The design philosophy will prioritize user-friendliness for PyTorch Geometric (PyG) and NetworkX users, satisfying JOSS's usability requirements. Below is a conceptual snippet of the API:

```
import torch
from torch_geometric.data import Data
from koopman_graph import GraphKoopmanModel, GNNEncoder, GNNDecoder

# 1. Define the network architecture
encoder = GNNEncoder(in_channels=3, hidden_channels=64)
decoder = GNNDecoder(hidden_channels=64, out_channels=3)

# 2. Initialize the Spatiotemporal Koopman Model
model = GraphKoopmanModel(
    encoder=encoder,
    decoder=decoder,
    latent_dim=64,
    time_step=0.1
)

# 3. Train on dynamic graph snapshots (PyG Data objects)
# data_sequence contains node features X_t and edge_index for t=0...T
model.fit(data_sequence, epochs=100)

# 4. Predict future graph states linearly in latent space
future_graphs = model.predict(initial_graph, steps=50)
```

4. Meeting JOSS Review Criteria

JOSS evaluates the engineering quality of the software rather than the mathematical novelty. The package will be designed from day one to satisfy JOSS's rigorous standards:

- **Comprehensive Documentation:** Hosted via Sphinx/ReadTheDocs, including a detailed API reference, installation instructions, and explicit community contribution guidelines (CONTRIBUTING.md).
- **Educational Tutorials:** A suite of Jupyter notebooks demonstrating end-to-end workflows on standard networked datasets (e.g., IEEE 118-bus system and traffic datasets) to satisfy the JOSS requirement for practical examples.
- **Testing & CI/CD:** A minimum of 80% test coverage using pytest, integrated via GitHub Actions to ensure cross-platform compatibility and stability on every pull request.
- **Community & Licensing:** Released under an OSI-approved license (e.g., MIT or Apache 2.0) with clear issue templates for bug reports and feature requests.

5. Development Roadmap

Phase	Milestone	Deliverables
Phase 1 (Months 1-2)	Core Architecture & PyG Integration	Standard GCN/GAT encoders, basic linear propagator, initial unit tests.
Phase 2 (Months 3-4)	Loss Optimization & Stability	Forward/backward consistency losses, expanding test suite to >80% coverage.
Phase 3 (Months 5-6)	JOSS Tutorials & Documentation	Jupyter notebook tutorials (Smart grids, traffic), Sphinx docs, CONTRIBUTING.md.

Phase	Milestone	Deliverables
Phase 4 (Months 7-8)	JOSS Paper Submission & Release	Draft paper.md, finalize CI/CD pipeline, PyPI publication, submit to JOSS.

6. Conclusion

By adhering to the rigorous software engineering standards expected by the *Journal of Open Source Software*, this package will transition complex Graph Koopman theoretical math into a highly accessible, tested, and reliable tool. Publishing in JOSS will formalize its contribution to the open-source community, ensuring it becomes a foundational asset for academic and enterprise researchers studying networked dynamics.